

SIGMA: The ‘SIGn-and-Mac’ Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols

Hugo Krawczyk

EE Department, Technion, Haifa, Israel, and IBM T.J. Watson Research Center.
`hugo@ee.technion.ac.il`

Abstract. We present the SIGMA family of key-exchange protocols and the “SIGn-and-Mac” approach to authenticated Diffie-Hellman underlying its design. The SIGMA protocols provide perfect forward secrecy via a Diffie-Hellman exchange authenticated with digital signatures, and are specifically designed to ensure sound cryptographic key exchange while providing a variety of features and trade-offs required in practical scenarios (such as optional identity protection and reduced number of protocol rounds). As a consequence, the SIGMA protocols are very well suited for use in actual applications and for standardized key exchange. In particular, SIGMA serves as the cryptographic basis for the signature-based modes of the standardized Internet Key Exchange (IKE) protocol (versions 1 and 2).

This paper describes the design rationale behind the SIGMA approach and protocols, and points out to many subtleties surrounding the design of secure key-exchange protocols in general, and identity-protecting protocols in particular. We motivate the design of SIGMA by comparing it to other protocols, most notable the STS protocol and its variants. In particular, it is shown how SIGMA solves some of the security shortcomings found in previous protocols.

1 Introduction

In this paper we describe the SIGMA family of key-exchange protocols, with emphasis on its design features and rationale. The SIGMA protocols introduce a general approach to building authenticated Diffie-Hellman protocols using a *careful* combination of digital signatures and a MAC (message authentication) function. We call this the “*SIGn-and-Mac*” approach which is also the reason for the SIGMA acronym.

SIGMA serves as the cryptographic basis for the Internet Key Exchange (IKE) protocol [11,16] standardized to provide key-exchange functionality to the IPsec suite of security protocols [17]. More precisely, SIGMA is the basis for the signature-based authenticated key exchange in IKE [11], which is the most commonly used mode of public-key authentication in IKE, and the basis for the only mode of public-key authentication in IKEv2 [16].

This paper provides the first systematic description of the development and rationale of the SIGMA protocols. The presentation is intended to motivate the design choices in the protocol by comparing and contrasting it to alternative protocols, and by learning from the strong and weak aspects of previous protocols. It also explains how the different variants of the SIGMA protocol follow from a common design core. In particular, it explains the security basis on which the signature-based modes of IKE, and its current revision IKEv2, are based. The presentation is informal and emphasizes rationale and intuition rather than rigorous analysis. A formal analysis of the SIGMA protocol has been presented in [7] where it is shown that the basic SIGMA design and its variants are secure under a complexity-theoretic model of security. While this rigorous analysis is essential for gaining confidence in the security design of SIGMA, it does not provide an explicit understanding of the design process that led to these protocols, and the numerous subtleties surrounding this design. Providing such an understanding is a main goal of this paper which will hopefully be beneficial to cryptographers and security protocol designers (as well as for those engineering security solutions based on these protocols).

The basic guiding requirements behind the design of SIGMA are (a) to provide a secure key-exchange protocol based on the Diffie-Hellman exchange (for ensuring “perfect forward secrecy”), (b) use digital signatures as the means for public-key authentication of the protocol, and (c) provide the option to protect the identities of the protocol peers from being learned by an attacker in the network. These were three basic requirements put forth by the IPsec working group for its preferred key-exchange protocol. The natural candidate for satisfying these requirements is the well-known STS key-exchange protocol due to Diffie, van Oorschot and Wiener [8]. We show, however, that this protocol and some of its variants (including a variant adopted into Photuris [14], a predecessor of IKE as the key-exchange protocol for IPsec) suffer from security shortcomings that make them unsuited for some practical scenarios, in particular in the wide Internet setting for which the IPsec protocols are designed. Still, the design of SIGMA is strongly based on that of STS: both the strengths of the STS design principles (very well articulated in [8]) as well as the weaknesses of some of the STS protocol choices have strongly influenced the SIGMA design.

One point that is particularly important for understanding the design of SIGMA (and other key-exchange protocols) is the central role that the requirement for identity protection has in this design. As it turns out, the identity protection functionality conflicts with the essential requirement of peer authentication. The result is that both requirements (authentication and identity protection) can be satisfied simultaneously, at least to some extent, but their co-existence introduces significant subtleties both in the design of the protocol and its analysis. In order to highlight this issue we compare SIGMA to another authenticated Diffie-Hellman design, a variant of the ISO protocol [12], that has been shown to be secure [6] but which is not well-suited to support identity protection. As we will see SIGMA provides a satisfactory and flexible solution to this problem by supporting identity protection as an optional feature of the pro-

protocols, while keeping the number of communication rounds and cryptographic operations to a minimum. As a result SIGMA can suit the identity protection scenarios as well as those that do not require this functionality; in the later case there is no penalty (in terms of security, computation, communication, or general complexity) relative to protocols that do not offer identity protection at all. We thus believe that SIGMA is well suited as a “general purpose” authenticated Diffie-Hellman protocol that can serve a wide range of applications and security scenarios.

History of the SIGMA protocols. The SIGMA approach was introduced by the author in 1995 [19] to the IPsec working group as a possible replacement for the Photuris key-exchange protocol [14] developed at the time by that working group. Photuris used a variant of the STS protocol that we showed [19] to be flawed through the attack presented in Section 3.3. In particular, this demonstrated that the Photuris key exchange, when used with optional identity protection and RSA signatures (or any signature scheme allowing for message recovery), was open to the same attack that originally motivated the design of STS (see Section 3.1). Eventually, the Photuris protocol was replaced with the Internet Key Exchange (IKE) protocol which adopted SIGMA (unnamed at the time) into its two signature-based authentication modes: main mode (that provides identity protection) and aggressive mode (which does not support identity protection). The IKE protocol was standardized in 1999, and a revised version (IKEv2) is currently under way [16] (the latter also uses the SIGMA protocol as its cryptographic key exchange).

Related work. There is a vast amount of work that deals with the design and analysis of key-exchange (and authentication) protocols and which is relevant to the subject of this paper. Chapter 12 of [27] provides many pointers to such works, and additional papers can be found in the recent security and cryptography literature. There have been a few works that provided analysis and critique of the IKE protocol (e.g., [9,29]). Yet, these works mainly discuss issues related to functionality and complexity trade-offs rather than analyzing the core cryptographic design of the key exchange protocols. A formal analysis of the IKE protocols has been carried by Meadows [26] using automated analysis tools. In addition, as we have already mentioned, [7] provides a formal analysis of the SIGMA protocols (and its IKE variants) based on the complexity-theoretic approach to the analysis of key-exchange protocols initiated in [2]. A BAN-logic analysis of the STS protocols is presented in [31], and attacks on these protocols that enhance those reported in [19] are presented in [4] (we elaborate on these attacks in Section 3.3). Finally, we mention the SKEME protocols [20] which served as the basis for the cryptographic structure of IKE and its non-signature modes of authentication, but did not include a signature-based solution as in SIGMA.

Organization. In Section 2 we informally discuss security requirements for key-exchange protocols in general and for SIGMA in particular, and present specific requirements related to identity protection. Section 3 presents the STS protocol and its variants, and analyzes the strengths and weaknesses of these

protocols. Section 4 discusses the ISO protocol as a further motivation for the design of SIGMA (in particular, this discussion serves to stress the role of identity protection in the design of SIGMA). Finally, Section 5 presents the SIGMA protocols together with their design rationale and security properties. In particular, Section 5.4 discusses the SIGMA variants used in the IKE protocols.

Note: In the full version of this paper [24] some additional information is provided as appendices. Specifically, Appendix A includes a simplified (and somewhat informal) definition of key-exchange security. Appendix B presents a “full fledge” instantiation of SIGMA which includes some of the elements omitted in the simplified presentation of Section 5 but which are crucial for a full secure implementation of the protocols. Appendix C discusses key-derivation issues and presents the specific key-derivation technique designed for, and used in, the IKE protocols. This technique is of independent interest since it applies to the derivation of keys in other key-exchange protocols; in particular, it includes a mechanism for “extracting randomness” from Diffie-Hellman keys.

2 Preliminaries: On the Security of Key-Exchange Protocols

Note: This section is important for understanding the design goals of SIGMA; yet, the impatient reader may skip it in a first reading (but see the notation paragraph at the end of the section).

In this paper we present an informal exposition of the design rationale behind the development of the SIGMA protocols. This exposition is intended to serve crypto protocol designers and security engineers to better understand the subtle design and analytical issues arising in the context of key-exchange (KE for short) protocols in general, and in the design of SIGMA in particular. This exposition, however, is not a replacement for a formal analysis of the protocol. A serious analysis work requires a formal mathematical treatment of the underlying security model and protocol goals. This essential piece of work for providing confidence in the security of the SIGMA protocols is presented in a companion paper [7]. The interested reader should consult that work for the formal foundations of security on which SIGMA is based. Yet, before going on to present the SIGMA protocols and some of its precursors we discuss informally some of the salient aspects of the analytical setting under which we study and judge KE protocols. This presentation will also provide a basis for the discussion of some of the techniques, strengths and weaknesses showing up in the protocols studied in later sections.

We start by noting that *there is no ultimate security model*. Security definitions may differ depending on the underlying mathematical methodology, the intended application setting, the consideration of different properties as more or less important, etc. The discussion below focuses on the core security properties of KE protocols as required in most common settings. These requirements stem from the quintessential application of KE protocols, namely, the supply of shared keys to pairs of parties which later use these keys to secure (via

integrity and secrecy protection) their pairwise communications. In addition, we deal with some more specific design goals of SIGMA motivated by requirements put forth by the IPsec working group: the use of the Diffie-Hellman exchange as the basic technique for providing “perfect forward secrecy”, the use of digital signatures for authenticating the exchange, and the (possibly optional) provision of “identity protection”.

2.1 Overview of the Security Model and Requirements

In spite of being a central (and “obvious”) functionality in many cryptographic and security applications, the notion of a “secure key-exchange protocol” remains a very complex notion to formalize correctly. Here we state very informally some basic requirements from KE protocols that we will use as a basis for later discussion of security issues arising in the design of KE protocols. These requirements are in no way a replacement for a formal treatment carried in [7], but are consistent (at least at the intuitive level) with the notion of security in that work.

Authentication. Each party to a KE execution (referred to as a *session*) needs to be able to uniquely *verify the identity* of the peer with which the session key is exchanged.

Consistency. If two honest parties establish a common session key then both need to have a consistent view of who the peers to the session are. Namely, if a party A establishes a key K and believes the peer to the exchange to be B , then if B establishes the session key K then it needs to believe that the peer to the exchange is A ; and vice-versa.

Secrecy. If a session is established between two honest peers then no third party should be able to learn any information about the resultant session key (in particular, no such third party, watching or interfering with the protocol run, should be able to distinguish the session key from a random key).

While the “authentication” and “secrecy” requirements are very natural and broadly accepted, the requirement of “consistency” is much trickier and many times overlooked. In Section 3.1 we exemplify this type of failure through an attack first discovered in [8]. This attack, to which we refer as an “identity misbinding attack”, applies to many seemingly natural and intuitive protocols. Avoiding this form of attack and guaranteeing a consistent binding between a session key and the peers to the session is a central element in the design of SIGMA.

One important point to observe is that the above requirements are not absolute but exist only in relation to a well-defined attack model. The adversarial model from [7] assumes that each party holds a long-term private authentication key that is used to uniquely identify and authenticate this party (in the context of this paper we can concretely think of this long-term authentication key as being a secret digital signature key.) It also assumes the existence of a trusted certification authority, or any other trusted mechanism (manual distribution, web of trust, etc), for binding identities with public keys. Parties communicate

over a public network controlled by a fully-active man-in-the-middle attacker which may intercept, delete, delay, modify or inject messages at will. This attacker also controls the scheduling of protocol sessions (a *session* is an execution instance of the protocol) which may run concurrently at the same or different parties.

In addition, the attacker may learn some of the secret information held by the parties to the protocol. Specifically, the attacker may learn the long-term secret information held by a party, in which case this party is considered as controlled by the attacker (and referred to as *corrupted*). There is no requirement about the security of sessions executed by a corrupted party (since the attacker may impersonate it at will), however, it is required that session keys produced (and erased from memory) before the party corruption happened will remain secure (i.e. no information on these keys should be learned by the attacker). This protection of past session keys in spite of the compromise of long-term secrets is known as **perfect forward secrecy (PFS)** and is a fundamental property of the protocols discussed here. The attacker may also learn session-specific information such as the value of a session key or some secret information contained in the internal state of a session (e.g., the exponent x of an ephemeral Diffie-Hellman exponential g^x used in that session). In this case, there is no requirement on the security of the compromised session but we do require that this leakage has no effect on other (uncompromised) sessions. This models resistance to a variety of attacks, including **known-key attacks** and **replay attacks** (see [27]), and emphasizes the need for **key independence** between different sessions.

The analysis of protocols under this model is carried on the basis of the generic properties required from the cryptographic primitives used in the protocol, rather than based on the properties of specific algorithms. This **algorithm independence** (or **generic security**) principle is important in case that specific crypto algorithms need to be replaced (for better security or improved performance), and to support different combinations of individually secure algorithms.

Discussion: sufficiency of the above security requirements. One important question is whether the above security requirements (and more precisely the formal security requirements from [7]), under which we judge the security of protocols in this work, are necessary and/or sufficient to guarantee “key-exchange security”. Necessity is easy to show through natural examples in which the removal of any one of the above required properties results in explicit and clearly harmful attacks against the security of the exchanged key (either by compromising the secrecy of the key or by producing an inconsistent binding between the key and the identities of the holders of that key). Sufficiency, however, is harder to argue. We subscribe to the approach put forth in [6] (and followed by [7]) by which a minimal set of requirements for a KE protocol must ensure the security of the quintessential application of KE protocols, namely, the provision of “secure channels” (i.e., the sharing of a key between peers that subsequently use this key for protecting the secrecy and integrity of the information transmitted between them). It is shown in [7] that their definition (outlined here) is indeed sufficient (and actually minimalistic) for providing secure channels.

Also important to stress is that this definitional approach dispenses of some requirements that some authors (e.g., [25]) consider vital for a sound definition of security. One important example is the **aliveness requirement**, namely, if A completes a session with peer B then A has a proof that B was “alive” during the execution of the protocol (e.g., by obtaining B ’s unique authentication on some nonce freshly generated by A). This property is *not* guaranteed by our (or [7]) definition of security. Moreover, some natural key-transport protocols (e.g., the ENC protocol formally specified in [6]) are useful key-exchange protocols that guarantee secure channels yet do not provide a proof of aliveness. The only possible negative aspect of a KE protocol that lacks the aliveness guarantee is that a party may establish a session with a peer that did not establish the corresponding session (and possibly was not even operational at the time); this results in a form of “denial of service” for the former party but not a compromise of data transmitted and protected under the key. However, DoS attacks with similar effects are possible even if aliveness guarantees are provided, for example by the attacker preventing the arrival of the last protocol message to its destination.

A related (and stronger) property not guaranteed by our basic definition of security is **peer awareness**. Roughly speaking, a protocol provides peer awareness for A if when A completes a session with peer B , A has a guarantee that (not only is B alive but) B has initiated a corresponding session with peer A . Adding aliveness and peer awareness guarantees to a KE that lacks these properties is often very simple, yet it may come at a cost (e.g., it may add messages to the exchange or complicate other mechanisms such as identity protection). Therefore, it is best to leave these properties as optional rather than labeling as “insecure” any protocol that lacks them.¹

All the protocols discussed in this paper provide aliveness proofs to both parties but only the ISO protocol and the 4-message SIGMA-I with added ACK (Section 5.2) provide peer awareness to both parties. In particular, the IKE protocols (Section 5.4) do not provide peer awareness to one of the peers. As said, this property can be added, when required, at the possible expense of extra messages or other costs.

2.2 Identity Protection

As discussed in Section 2.1, key-exchange protocols require strong mutual authentication and therefore they must be designed to communicate the identity of each participant in the protocol to its session peer. This implies that the identities must be transmitted as part of the protocol. Yet some applications require to prevent the disclosure of these identities over the network. This may be the case in settings where the identity (for the purpose of authentication) of a party is not directly derivable from the routing address that must appear in the clear in the protocol messages. A common example is the case of mobile

¹ We stress that in contrast to the key-exchange setting, the aliveness requirements, and sometimes peer awareness, is essential in “entity authentication” protocols whose sole purpose may be to determine the aliveness of a peer.

devices wishing to prevent an attacker from correlating their (changing) location with the logical identity of the device (or user). Note that such an application may not just need to hide these identities from passive observers in the network but may require to conceal the identity even from active attackers. In this case the sole encryption of the sender’s identity is not sufficient and it is required that the peer to the session proves its own identity before the encrypted identity is transmitted. As it turns out the requirement to support identity protection adds new subtleties to the design of KE protocols; these subtleties arise from the conflicting nature of identity protection and authentication. In particular, *it is not possible* to design a protocol that will protect both peer identities from active attacks. This is easy to see by noting that the first peer to authenticate itself (i.e. to prove its identity to the other party) must disclose its identity to the other party before it can verify the identity of the latter. Therefore the identity of the first-authenticating peer cannot be protected against an active attacker. In other words, KE protocols may protect both identities from passive attacks and may, at best, protect the identity of one of the peers from disclosure against an active attacker.

This best-possible level of identity protection is indeed achievable by some KE protocols, and in particular is attained by the SIGMA protocols. The underlying design of SIGMA allows for a protocol variant where the initiator of the exchange is protected against active attacks and the responder’s identity is protected against passive attacks (we refer to this variant as SIGMA-I), and it also allows for another variant where the responder’s id is protected against active attacks and the initiator’s against passive attacks only (SIGMA-R). Moreover, providing identity protection has been a main motivating force behind the design of SIGMA which resulted from the requirement put forth by the IPsec working group to support (at least optionally) identity protection in its KE protocol. The SIGMA protocols thus provide the best-possible protection against identity disclosure. The choice of SIGMA-I or SIGMA-R depends on which identity is considered as more sensitive and requires protection against active attacks. On the other hand, SIGMA also offers full KE security also in cases where identity protection is not needed. That is, the core security of the protocol does not depend on hiding identities but rather this protection of identities is a functionality added on top of the core protocol.

A related issue which is typical of settings where identity protection is a concern, but may also appear elsewhere, is that parties to the protocol may not know at the beginning of a session the specific identity of the peer but rather learn this identity as the protocol proceeds. (This may even be the case for the initiator of the session which may agree to establish the initiated session with one of a set of peers rather than with one predefined peer). This adds, in principle, more attack avenues against the protocol and also introduces some delicate formal and design issues (e.g., most existing formalisms of key-exchange protocols do assume that the peer identities are fixed and known from the start of the session). In [7] this more general and realistic setting is formalized under

the name of the **post-specified peer setting** and the SIGMA protocols are shown to be secure in this model. See [7] for the technical details.

Finally we comment on one additional privacy aspect of KE protocols. In some scenarios parties may wish to keep their privacy protected not only against attackers in the network but also to avoid leaving a “provable trace” of their communications in the hands (or disks) of the peers with which they communicate. A protocol such as ISO (see Section 4) in which each party to the protocol signs the peer’s identity is particularly susceptible to this privacy concern (since these signatures can serve to prove to a third party the fact that the communication took place). In the SIGMA protocols, however, this proof of communication is avoided to a large extent by not signing the peer’s identity, thus providing a better solution to this problem.

Note: Some may consider the non-repudiation property of a protocol such as ISO (Section 4) as an advantage. However, we consider that non-repudiation using digital signatures does not belong to the KE protocol realm but as a functionality that needs to be dealt with carefully in specific applications, and with full awareness of the signer to the non-repudiation consequences.

2.3 Further Remarks and Notation

Denial of Service. Key-exchange protocols (including SIGMA) open opportunities for Denial-of-Service attacks since the responder to an exchange is usually required to generate state and/or perform costly computations before it has the opportunity to authenticate the peer to the exchange. This type of attacks cannot be prevented in a strong sense but can be mitigated by using some fast-to-verify measures. One such technique has been proposed by Phil Karn [14] via the use of “cookies” that the responder to a KE protocol uses to verify that the initiator of the exchange is being able to receive messages directed to the IP address from which the exchange was initiated (thus preventing some form of trivial DoS attacks in which the attacker uses forged origin addresses, and also improving the chances to trace back a DoS attack). This and other techniques are orthogonal to the cryptographic details of the KE protocol and then can be adopted into SIGMA. In particular, recent proposals for revising IKE [16,1] incorporate Karn’s technique into SIGMA. Other forms of denial of service are possible (and actually unavoidable) such as an active attacker that prevents the completion of sessions or lets one party complete the session and the other not.

A word of caution. It is important to remark that all the protocols discussed in this paper are presented in their most basic form, showing only their cryptographic core. When used in practice it is essential to preserve this cryptographic core but also to take care of additional elements arising in actual settings. For example, if the protocol negotiates some security parameters or uses the protocol messages to send some additional information then the designers of such full-fledge protocol need to carefully expand the coverage of authentication also to these additional elements. We also (over) simplify the protocol presentation by omitting the explicit use of “session identifiers”: such identifiers are needed

for the run of a protocol in a multi-session setting where they are used to match incoming protocol messages with open KE sessions. Moreover, the binding of messages to specific session id’s is required for core security reasons such as preventing interleaving attacks. Similarly, nonces may need to be included in the protocol to ensure freshness of messages (e.g. to prevent replay attacks). In our presentation, however, these elements are omitted by over-charging the Diffie-Hellman exponentials used in the protocols with the additional functionality of also serving as session ids and nonces. For the level of conceptual discussion in this paper, simplifying the presentation by reducing the number of elements in the protocol is useful (and also in line with the traditional presentation of protocols in the cryptographic literature, in particular with [8]). But when engineering a real-world protocol we recommend to clearly separate the functionality of different elements in the protocol. For illustration purposes, we present a version of a “full fledge” SIGMA protocol in [24].

Notation. All the protocols presented here use the Diffie-Hellman exchange. We use the traditional exponential notation g^x where g is a group generator. However, all the treatment here applies to any group in which the Diffie-Hellman problem is hard. (A bit more precisely, groups in which the so called “Decisional Diffie-Hellman Assumption (DDH)” holds, namely, the infeasibility to distinguish between quadruples of the form (g, g^x, g^y, g^{xy}) and quadruples (g, g^x, g^y, g^z) where x, y, z are random exponents.) We use the acronym DH to denote Diffie-Hellman, and use the noun “exponential” for elements such as g^x and the word “exponent” for x . In the description of our protocols the DH group and generator g are assumed to be fixed and known in advance to the parties or communicated at the onset of the protocol (in the later case, the DH parameters need to be included in the information authenticated by the protocol).

Throughout the paper we will also use the notation $\{\dots\}_K$ to denote encryption of the information between the brackets under a symmetric encryption function using key K . Other cryptographic primitives used in the paper are a MAC (message authentication code) which is assumed to be unforgeable against chosen message attack by any adversary that is not provided the MAC key, and a digital signature scheme SIG assumed to be secure against chosen message attacks. By $\text{SIG}_A(msg)$ we denote the signature using A ’s private key on the message msg . The letters A and B denote the parties running a KE protocol, while Eve (or E) denotes the (active) attacker. We also use A, B, E to denote the identities used by these parties in the protocols.

3 The STS Protocols

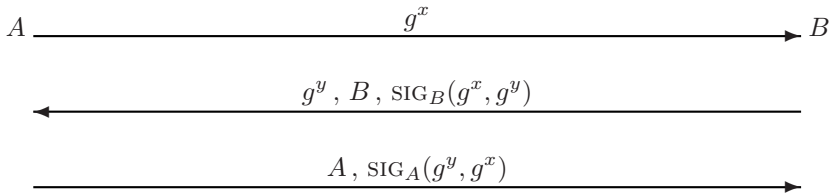
Here we discuss the STS protocol (and some of its variants) which constitutes one of the most famous and influential protocols used to provide authenticated DH using digital signatures, and of particular appeal to scenarios where identity protection is a concern. The STS protocol, due to Diffie, van Oorschot and Wiener, is presented in [8] where a very instructive description of its design rationale is provided. In particular, this work is the first to observe some of the

more intricate subtleties related to the authentication of protocols in general and of the DH exchange in particular. The STS protocol served as the starting point for the SIGMA protocols described in this paper. Both the strengths of the STS design principles as well as the weaknesses of some of the protocol choices have motivated the design of SIGMA. These aspects are important to be understood before presenting SIGMA. We analyze several variants of the protocol proposed in [8,27,14].

Remark. The attacks on the STS protocol and its variants presented here originate with the communications by the author to the IPsec working group in 1995 [19]. Since then some of these attacks were recalled elsewhere (e.g. [30]) and enhancements of the attack against the MAC variant have been provided in [4].

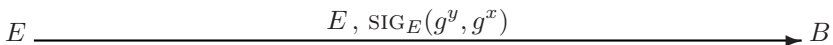
3.1 BADH and the Identity-Misbinding Attack: A Motivating Example

As the motivation for the STS protocol (and later for SIGMA too) we present a proposal for an “authenticated DH protocol” which intuitively provides an authenticated KE solution but is actually flawed. We denote this protocol by BADH (“badly authenticated DH”).



The output of the protocol is a session key K_s derived from the DH value g^{xy} . (Note: the identity of A may also be sent in the first message, this is immaterial to the discussion here.)

This protocol provides the most natural way to authenticate a DH exchange using digital signatures. Each party sends its DH exponential signed under its private signature key. The inclusion of the peer’s exponential under one’s signature is required to prove freshness of the signature for avoiding replay attacks (we will discuss more about this aspect in the context of SIGMA, in particular the possibility to replace the signature on the peer’s exponential with the signature on a peer-generated nonce). One of the important contributions of [8] was to demonstrate that this protocol, even if seemingly natural and intuitively correct, does not satisfy the important *consistency requirement* discussed in Section 2.1. Indeed, [8] present the following attack against the BADH protocol. An active (“person-in-the-middle”) attacker, which we denote by Eve (or E), lets the first two messages of the protocol to go unchanged between A and B , and then it replaces the third message from A to B with the following message from Eve to B :

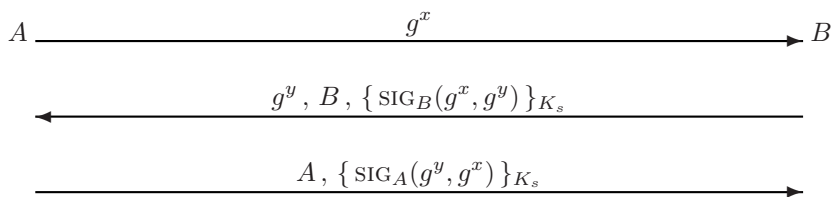


The result of the protocol is that A records the exchange of the session key K_s with B , while B records the exchange of the *same* key K_s with Eve. In this case, any subsequent application message arriving to B and authenticated under the key K_s will be interpreted by B as coming from Eve (since from the point of view of B the key K_s represents Eve not A). Note that this attack does not result in a breach of secrecy of the key (since the attacker does not learn, nor influence, the key in any way) but it does result in a severe breach of authenticity since the two parties to the exchange will use the same key with different understandings of who the peer to the exchange is, thus breaking the consistency requirement. To illustrate the possible adverse effects of this attack we use the following example from [8]: imagine B being a bank and A a customer sending to B a monetary element, such as an electronic check or digital cash, encrypted and authenticated under K_s . From the point of view of B this is interpreted as coming from Eve (which we assume to also be a customer of B) and thus the money is considered to belong to Eve rather than to A (hopefully for Eve the money will go to her account!).

The essence of the attack is that Eve succeeds in convincing the peers to the DH exchange (those that chose the DH exponentials) that the exchange ended successfully yet the derived key is bound by each of the parties to a different peer. Thus the protocol fails to provide an authenticated binding between the key and the honest identities that generated the key. We will refer to this attack against the consistency requirement of KE protocols as an **identity misbinding attack** (or just “misbinding attack” for short).²

3.2 The Basic STS Protocol

Having discovered the misbinding attack on the “natural” authenticated DH protocol BADH, Diffie et al. [8] designed the STS protocol intended to solve this problem. The basic STS protocol is:



where the notation $\{\dots\}_K$ denotes encryption of the information between the brackets under a symmetric encryption function using key K . In the STS protocol the key used for encryption is the same as the one output as the session key produced by the exchange³.

² This type of attack appears in the context of other authentication and KE protocols. It is sometimes referred to as the “unknown key share attack” [4,15].

³ This is a weakness of the protocol since the use of the session key in the protocol leaks information on the key (e.g., the key is not anymore indistinguishable from

Is this protocol secure? In particular, is the introduction of the encryption of the signatures sufficient to thwart the identity misbinding attack? This at least has been the intention of STS. The idea was that by using encryption under the DH key the parties to the exchange “prove” knowledge of this key something which the attacker cannot do. Yet, no proof of security of the STS protocol exists (see more on this below). Even more significantly we show here that the misbinding attack applies to this protocol in any scenario where parties can register public keys without proving knowledge of the corresponding signature key. (We note that while such “proof of possession” is required by some CAs for issuing a certificate, this is not a universal requirement for public key certificates; in particular it is not satisfied in many “out-of-band distribution” scenarios, webs of trust, etc.) In this case Eve can register A ’s public key as its own and then simply replace A ’s identity (or certificate) in the third message of STS with her own. B verifies the incoming message and accepts it as coming from Eve. Thus, in this case the STS protocol fails to defend against the misbinding attack. Thus, for the STS to be secure one must assume that a secure external mechanism for proof of possession of signature keys is enforced. As we will see both the ISO protocol discussed in Section 4 and the SIGMA protocols presented here do not require such a mechanism. Moreover, even under the assumption of external “proof of possession” the above STS protocol has not been proven secure.

Note. In [31] an analysis of the STS protocol based on an extension of BAN logic [5] is presented. However, the modeling of the encryption function in that analysis is as a MAC function. Therefore this analysis holds for the MAC variant of STS presented in the next subsection. However, as we will see, for considering that protocol secure one needs to assume that the CA verifies that the registrant of a public key holds the corresponding private key (proof of possession) and, moreover, that “on-line registration” attacks as discussed below are not possible.

What is the reason for this protocol failure? The main reason is to assume that the combination of proof of possession of the session key together with the signature on the DH exponentials provide a sufficient binding between the identities of the (honest) peers participating in the exchange and the resultant key. However, as the above attack shows this is not true in general. Can this shortcoming be corrected? One first observation is that encryption is not the right cryptographic function to use for proving knowledge of a key. Being able to encrypt a certain quantity under a secret key is no proof of the knowledge of that key. Such a “proof of key possession” is not guaranteed by common modes of encryption such as CBC and is explicitly violated by any mode using XOR of a (pseudo) random pad with the plaintext (such as counter or feedback

random). In addition, this can lead to the use of the same key with two different algorithms (one inside the KE protocol, and another when using the exchanged session key in the application that triggered the key exchange), thus violating the basic cryptographic principle of key separation (see, e.g., [20]). These weaknesses are easily solved by deriving different, and computationally independent, keys from the DH value g^{xy} , one used internally in the protocol for encryption and the other as the session key output by the protocol.

modes, stream ciphers, etc.). To further illustrate this point consider a seemingly stronger variant of the protocol in which not only the signature is encrypted but also the identity (or full certificate) of the signer is encrypted too. In this case the above attack against STS is still viable if the encryption is of the XOR type discussed above. In this case, when A sends the message $\{A, \text{SIG}_A(g^y, g^x)\}_{K_s}$, Eve replaces A ’s identity (or certificate) by just XORing the value $A \oplus E$ in the identity location in the ciphertext. When decrypted by B this identity is read as E ’s and the signature verified also as E ’s. Thus we see that even identity encryption does not necessarily prevent the attack. As we will see in the next section replacing the encryption with a MAC function, which is better suited to prove possession of a key, is still insufficient to make the protocol secure.

3.3 Two STS Variants: MACed-Signature and Photuris

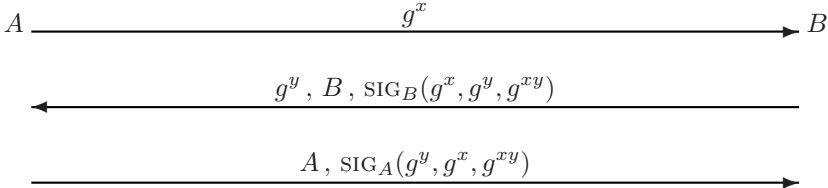
In [8] a variant of the basic STS protocol is suggested in which the encryption function in the protocol is replaced with a message authentication (MAC) function. Namely, in this STS variant each party in the protocol applies its signature on the DH exponentials plus it concatenates to it a MAC on the signature using the key K_s . For example, the last message from A to B in this protocol consists of the triple (A, b, c) where $b = \text{SIG}_A(g^y, g^x)$ and $c = \text{MAC}_{K_s}(b)$. In [8] this variant is not motivated as a security enhancement but as an alternative for situations –such as export control restrictions– in which the use of a strong encryption function is not viable. However, considering that a MAC function is more appropriate for “proving knowledge of a key” than an encryption function (as exemplified above) then one could expect that this variant would provide for a more secure protocol. This is actually incorrect too. The above attack on basic STS (where Eve records the public key of A under her name) can be carried exactly in the same way also in this MAC-based variant of the protocol. Same for the case where on top of the signature and identities (or even on top of the MAC) one applies an encryption function of the XOR type.

Moreover, if (as it is common in many application) A and B communicate their public key to each other as part of the KE protocol (i.e., the identities A and B sent in the protocol are their public-key certificates), then this MAC-ed signature variant is not secure even if the system does ensure that the registrant of a public key knows the corresponding private key! This has been shown by Blake-Wilson and Menezes [4] who present an ingenious *on-line registration attack* against the protocol. In this form of attack, the attacker Eve intercepts the last message from A to B and then registers a public key (for which she knows the private key) that satisfies $\text{SIG}_E(g^y, g^x) = \text{SIG}_A(g^y, g^x)$. Eve then replaces the certificate of A with her own in the intercepted message and forwards it to B (leaving the signature and mac strings unchanged from A ’s original message). Clearly, B will accept this as a valid message from Eve since both signature and mac will pass verification. In other words, Eve successfully mounted an identity-misbinding attack against the MACed-signature protocol. In [4] it is shown that this on-line registration attack can be performed against natural signature schemes. In particular, it is feasible against RSA signatures provided

that the registrant of the public key can choose her own public RSA exponent.⁴ While the full practicality of such an attack is debatable, it certainly suffices to show that one cannot prove this protocol to be secure on the basis of generic cryptographic functions, even under the assumption that the CA verifies possession of the private signature key. As a final note on this attack, we point out that this attack is possible even if the protocol is modified in such a way that each peer includes its own identity under the signature (something that can be done to avoid the need for “proof of possession” in the public-key registration stage).

From the above examples we learn that the failure to the misbinding attack is more essentially related to the *insufficiency* of binding the DH key with the signatures. Such a binding (e.g., via a MAC) provides a proof that *someone* knows the session key, but does not prove *who* this *someone* is. As we will see later, the essential binding here needs to be done between the signature and the recipient’s identity (the ISO protocol) or between the DH key and the sender’s identity (the SIGMA protocol).

We finish this section by showing the insecurity of another variant of the STS protocol described in [27] and used as the core cryptographic protocol in Photuris [14] (an early proposal for a KE protocol for IPsec). As the previous variants, this one is also illustrative of the subtleties of designing a good KE protocol. This variant dispenses of the use of encryption or MAC; instead it attempts at binding the DH key to the signatures by including the DH key g^{xy} under the signature:



An obvious, immediate, complaint about this protocol is that the DH key g^{xy} is included under the signature, and therefore any signature that leaks information on the signed data (for example, any signature scheme that provides “message recovery”) will leak information on g^{xy} . This problem is relatively easy to fix: derive two values from g^{xy} using a one-way pseudorandom transformation; use one value to place under the signature, and the other as the generated session key. A more subtle weakness of the protocol is that it allows, even with the above enhancement, for an identity misbinding attack whenever the signature scheme allows for message recovery (e.g. RSA). In this case the attacker, Eve, proceeds as follows: it lets the protocol proceed normally between A and B for the first two messages, then it intercepts the last message from A to B and replaces it with the message

⁴ In this case, Eve uses an RSA public modulus equal to the product of two primes p and q for which computing discrete logarithms is easy (e.g., all factors of $p - 1$ and $q - 1$ are small), and calculates the private exponent d for which $(hash(g^y, g^x))^d$ equals the signature sent by A .

$$E \xrightarrow{E, \text{SIG}_E(g^y, g^x, g^{xy})} B$$

But how can E sign the key g^{xy} (or a value derived from it) if it does not know g^{xy} ? For concreteness assume that $\text{SIG}_A(M) = \text{RSA}_A(\text{hash}(M))$, for some hash (and encoding) function *hash*. Since Eve knows *A*’s public key it can invert *A*’s signature to retrieve $\text{hash}(g^y, g^x, g^{xy})$ and then apply its own signature $\text{RSA}_E(\text{hash}(g^y, g^x, g^{xy}))$ as required to carry the above attack! (Note that this attack does not depend on any of the details of the public-key registration process; the attacker uses its legitimately generated and registered public key.)

Photuris included the above protocol as an “authentication only” solution, namely, one in which identities are not encrypted. It also offered optional identity protection by applying encryption on top of the above protocol. In the later case the above simple misbinding attack does not work. Yet, even in this case no proof of security for such a protocol is known. The above protocol (without encryption) is also suggested as an STS variant in [27] where it is proposed to explicitly hash the value g^{xy} before including it under the signature.

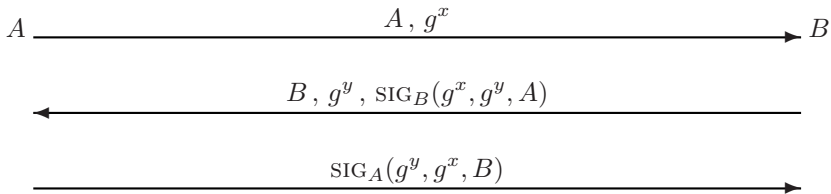
Remark: In this STS variant [27] the value g^{xy} under the signature is replaced with $h(g^{xy})$ where *h* is a hash function. This explicit hashing of g^{xy} seems to be intended to protect the value g^{xy} in case that the signature in use reveals its input. While this is not sufficient to defend against our identity misbinding attack, it is interesting to check whether revealing the value $h(g^{xy})$ may be of any use to an eavesdropper (note that in this case the attacker has the significantly simpler task of monitoring the protocol’s messages rather than actively interfering with the protocol as required to carry the misbinding attack). Certainly, learning $h(g^{xy})$ is sufficient for distinguishing the key g^{xy} from random (even if the hash function acts as an ideal “random oracle”). But can the attacker obtain more than that? To illustrate the subtle ways in which security deficiencies may be exploited, consider the following practical scenario in which the function *h* is implemented by SHA-1 and the key derivation algorithm defines the session key to be $K_s = \text{HMAC-SHA1}_{g^{xy}}(v)$, where *v* is a non-secret value. The reader can verify (using the definition of HMAC in [21]) that in this case the attacker does not need to find g^{xy} for deriving the session key K_s , but it suffices for her to simply know $\text{SHA-1}(g^{xy})$. Therefore if this later value is revealed by the signature then the security of the protocol is totally lost. Not only this example shows the care required in designing these protocols, but it also points to the the potential weaknesses arising from protocols whose security cannot be claimed in a generic (i.e. algorithm-independent) way.

4 The ISO Protocol

Here we recall the ISO KE protocol [12] which similarly to STS uses digital signatures to authenticate a DH exchange⁵. However, the ISO protocol resolves the problem of key-identity binding demonstrated by the misbinding attack on

⁵ Strictly speaking, the protocol presented here is a simplification of the protocol in [12]. The latter includes two elements that are redundant and do not contribute

the BADH protocol (see Section 3.1) differently. The protocol simply adds the identity of the *intended recipient* of the signature to the signed information. Specifically, the protocol is:



It is not hard to see that the *specific* identity misbinding attack as described in Section 3.1 is avoided by the inclusion of the identities under the signatures. Yet having seen the many subtleties and protocol weaknesses related to the STS protocols in the previous section it is clear that resolving one specific attack is no guarantee of security. Yet the confidence in this protocol can be based on the analytical work of [6] where it is shown that this is a secure KE protocol (under the security model of that work). It is shown there that any feasible attack in that model against the security of the ISO protocol can be transformed into an efficient cryptanalytical procedure against the DH transform or against the digital signature function scheme in use.

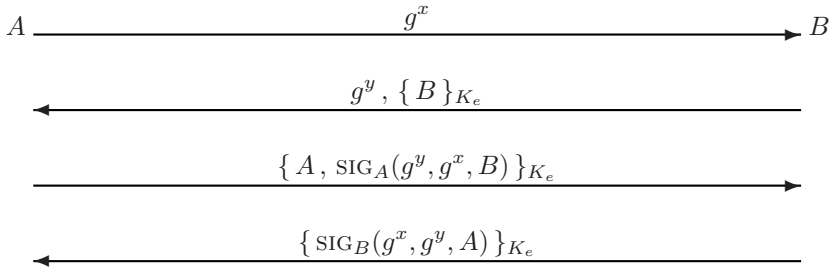
The ISO protocol is simple and elegant. It uses a minimal number of messages and of cryptographic primitives. It allows for delaying computation of the DH key g^{xy} to the end of the interaction (since the key is not used inside the protocol itself) thus reducing the effect of computation on protocol latency. The protocol is also minimal in the sense that the removal of any of its elements would render the protocol insecure. In particular, as demonstrated by the BADH protocol, the inclusion of the recipient's identity under the signature is crucial for security. It is also interesting to observe that replacing the recipient's identity under the signature with the signer's identity results in an insecure protocol, open to the identity-misbinding attack exactly as in the case of BADH.

Therefore, it seems that we have no reason to look for other DH protocols authenticated with digital signatures. This is indeed true as long as "identity protection" is not a feature to be supported by the protocol. As explained below, in spite of all its other nice properties the ISO protocol does *not* satisfactorily accommodate the settings in which the identities of the participants in the protocol are to be concealed from attackers in the network (especially if such a protection is sought against active attacks).

The limitation of the ISO protocol in providing identity protection comes from the fact that in this protocol each party needs to know the identity of the peer before it can produce its own signature. This means that no party to the

significantly to the security of the protocol and are therefore omitted here. These elements are the inclusion of the signer's identity under the signature and an additional MAC value. In contrast to SIGMA, where the additional MAC is essential for security, the MAC in [12] serves only for explicit key confirmation (which adds little to the implicit key confirmation provided in the simplified variant discussed here).

protocol (neither A or B) can authenticate the other party before it reveals its own name to that party. This leaves *both* identities open to active attacks. If the only protection sought in the protocol is against passive eavesdroppers then the protocol can be built as a 4-message protocol as follows:



where K_e is an encryption key derived from the DH key g^{xy} . We note that with this addition of encryption the ISO protocol loses several of its good properties (in particular, the minimality discussed above and the ability to delay the computation of g^{xy} to the end of the protocol) while it only provides partial protection of identities since both identities are trivially susceptible to active attacks.

Another privacy (or lack of privacy) issue related to the ISO protocol which is worth noting is that by signing the peer’s identity each party to the protocol leaves in the hands of the peer a signed (undeniable) trace that the communication took place (see the discussion at the end of Section 2.2).

The SIGMA protocol presented in the next section provides better, and more flexible, support for identity protection with same or less communication and computational cost, and with an equivalent proof of security.

Remark (*an identity-protection variant of the ISO protocol*): We end this section by suggesting an adaptation of the ISO protocol to settings requiring identity protection (of one of the peers) to active attacks. We only sketch the idea behind this protocol. The idea is to run the regular ISO protocol but instead of A sending its real identity in the first message it sends an “alias” computed as $\hat{A} = \text{hash}(A, r)$ for a random r . Then B proceeds as in the basic protocol but includes the value $\hat{A}$ under its signature instead of A ’s identity; it also uses the key g^{xy} to encrypt its own identity and signature. In the third message A reveals its real identity ‘ A ’ and the value r used to compute $\hat{A}$. It also sends its signature (with B ’s identity signed as in the regular ISO protocol). This whole message is privacy-protected with encryption under K_e . The above protocol can be shown to be secure under certain assumptions on the hash function *hash*. Specifically, this function needs to satisfy some “commitment” properties similar to those presented in [22].

We omit further discussion of this protocol and proceed to present the SIGMA protocol that provides a satisfactory and flexible solution to the KE problem suitable also for settings with identity protection requirements, and with less requirements on the underlying cryptographic primitives than the above “alias-based” ISO variant.

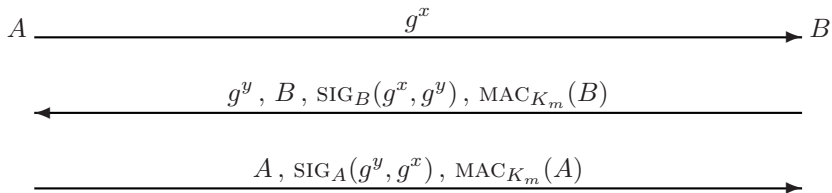
5 The SIGMA Protocols

The weaknesses of the STS variants (which provide identity protection but not full security in general) and the unsuitability of the ISO protocol for settings where identity protection is a requirement motivated our search for a solution that would provide solid security for settings where identity protection is or is not a requirement. The result is the SIGMA protocols that we present here and whose design we explain based on the design lessons learned through the examples in previous sections (and many other in the literature). SIGMA takes from STS the property that each party can authenticate to the other without needing to know the peer's identity (recall that the lack of this property in the ISO protocol makes that protocol inappropriate to support identity protection). And it takes from ISO the careful binding between identities and keys, but it implements this binding in a very different way. More specifically, SIGMA decouples the authentication of the DH exponentials from the binding of key and identities. The former authentication task is performed using digital signatures while the latter is done by computing a MAC function keyed via g^{xy} (or more precisely, via a key derived from g^{xy}) and applied to the sender's identity. This "SIGn-and-MAC" approach is the essential technique behind the design of these protocols and the reason for the SIGMA acronym.

As pointed out in Section 2.3, we focus on the cryptographic core of the protocol leaving important system and implementation details out of the discussion. In particular, as we will also note below, in the following presentation we overcharge the DH exponentials with the added functionality of session id's and freshness nonces. (A "full fledged" SIGMA instantiation with a more careful treatment of these elements is presented in [24].)

5.1 The Basic SIGMA Protocol

The most basic form of SIGMA (without identity protection) is the following:



The output of the protocol is a session key K_s derived from the DH value g^{xy} while the key K_m used as a MAC key in the protocol is also derived from this DH value. It is essential for the protocol security that the keys K_m and K_s be "computationally independent" (namely no information on K_s can be learned from K_m and vice-versa).⁶ Note that this basic protocol does not provide identity protection. This will be added on top of the above protocol using encryption

⁶ We discuss specific ways to derive these values from g^{xy} using pseudorandom functions in the full version of this paper [24].

(see following sections). The important point is that SIGMA’s security is built in a modular way such that its core cryptographic security is guaranteed independently of the encryption of identities. Thus the same design serves for scenarios requiring identity protection but also for the many cases where such protection is not an issue (or is offered only as an option). We note that the identities A and B transmitted in messages 2 and 3 may be full public-key certificates; in this case the identities included under the MAC may be the certificates themselves or identities bound to these certificates.

The first basic element in the logic of the protocol is that the DH exponential chosen by each party is protected from modification (or choice) by the attacker via the signature that the party applies to its own exponential. We note that the inclusion of the peer’s exponential under the signature is not mandatory and can be replaced with a nonce freshly chosen and communicated by the peer. Yet, either the peer’s exponential (if chosen fresh and anew in each session) or a fresh nonce must be included under the signature; otherwise the following *replay attack* is possible. It would suffice for the attacker to learn the exponent x of a single ephemeral exponential g^x used by a party A in one session for the attacker to be able to impersonate A on a KE with any other party (simply by replaying the values g^x and $\text{SIG}_A(g^x)$). Thus, in this case A ’s impersonation by the attacker is possible even without learning A ’s long-term signature key. This violates the security principle (see Section 2.1) by which the exposure of ephemeral secrets belonging to a specific session should not have adverse effects on the security of other sessions.

The second fundamental element in SIGMA’s design is the MACing of the sender’s identity under a key derived from the DH key. This can be seen as a “proof of possession” of the DH key but its actual functionality is to *bind* the session key to the identity of each of the protocol participants in a way to provide the “consistency” requirement of KE protocols. As discussed in Section 2.1, this is a fundamental requirement needed, in particular, to avoid attacks such as the identity misbinding attack from Section 3.1. Note that without this MACing the protocol “degenerates” into the BADH protocol from Section 3.1 which is susceptible to this attack. Therefore we can see that all the elements in the protocol are mandatory (up to replacement of the peer’s exponential under the signature with a fresh nonce).

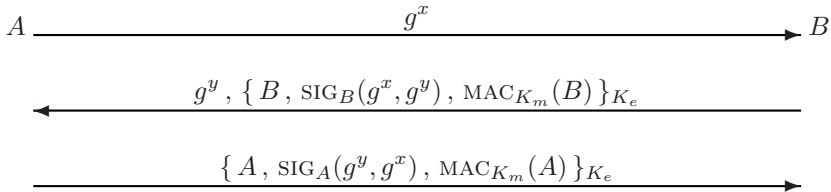
We note that the above SIGMA protocol, as well as all the following variants, satisfy all the security guarantees discussed in Section 2.1. In particular, they provide “perfect forward secrecy (PFS)” due to the use of the Diffie-Hellman exchange. This assumes that DH exponentials are chosen anew and independently for each session, that the exponents x, y used in a DH exponentials g^x, g^y are erased as soon as the computation of the key g^{xy} is completed, and that these exponents are not derivable from any other quantity stored in the party’s computer (in particular, if x is generated pseudorandomly then the value of past exponents x should not be derivable from the present state of the PRG). We note that SIGMA can allow for re-use of DH exponentials by the same party across different sessions. However, in this case the forward secrecy property is

lost (or at least confined to hold only after all sessions using the exponent x are completed and the exponent x erased). In case of re-use of DH exponents one must derive the keys used by the session (e.g. K_m , K_s) in a way that depends on some session-specific non-repeating quantity (a nonce or session-id). Also, as discussed before, in this case such a fresh nonce needs to be included under the peer's signature. There are other, more theoretical, issues concerning the re-use of DH exponents that are not treated here.

As we have stressed before, this informal outline of the design rationale for SIGMA does not constitute a proof of security for the protocol. The formal analysis in which we can base our confidence in the protocol appears in the companion analysis paper [7].

5.2 Protecting Identities: SIGMA-I

As said, SIGMA is designed to serve as a secure key-exchange protocol both in settings that do not require identity protection (in which case the above simple protocol suffices) or those where identity protection is a requirement. **The main point behind SIGMA's design that allows for easy addition of identity protection is that the peer's identity is not needed for own authentication.** In particular, one of the peers can delay communicating its own identity until it learns the peer's identity in an authenticated form. Specifically, to the basic SIGMA protocol we can add identity protection by simply encrypting identities and signatures using a key K_e derived from g^{xy} (K_e must be computationally independent from the authentication key K_m and the session key K_s):



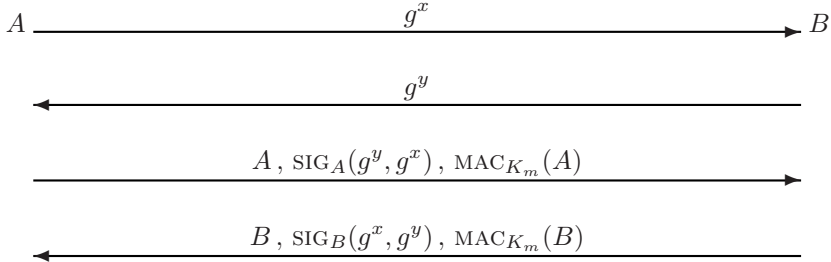
This protocol has the property that it protects the identity of the initiator from active attackers and the identity of the responder from passive attackers. Thus, the protocol is suitable for situations where concealing the identity of the initiator is considered of greater importance. A typical example is when the initiator is a mobile client connecting to a remote server. There may be little or no significance in concealing the server's identity but it may be of prime importance to conceal the identity of the mobile device or user. We stress that the encryption function (as applied in the third message) must be resistant to active attacks and therefore must combine some form of integrity. Combined secrecy-integrity transforms such as those from [13] can be used, or a conventional mode of encryption (e.g. CBC) can be used with a MAC function computed on top of the ciphertext [3, 23]. Due to the stronger protection of the identity of the Initiator of the protocol we denote this variant by SIGMA-I.

We remark that while this protocol has the minimal number of messages that any KE protocol resistant to replay attacks (and not based on trusted

timestamps) can use, it is sometimes desirable to organize the protocol in full round-trips with each pair of message containing a “request message” and a “response message”. If so desired, the above protocol can add a fourth message from B to A with a simple ACK authenticated under the authentication key K_m . This ACK message serves to A as a proof that B already established the key and communications protected under the exchanged key K_s can start. It also provides the flexibility for A to either wait for the ACK or start using the session key as soon as it sent the third protocol message. (Depending on B ’s policy this traffic may be accepted by B if the channel – or “security association” in the language of IKE – was already established by B , or discarded if not, or queued until the key establishment is completed.) Finally, it is worth noting that this ACK-augmented protocol provides the *peer awareness* property discussed in Section 2.1. (This is in contrast to the other variants of SIGMA presented here which do not enjoy this property.)

5.3 A Four Message Variant: SIGMA-R

As seen, SIGMA-I protects the initiator’s identity against active attacks and the responder’s against passive attacks. Here we present SIGMA-R which provides defense to the responder’s identity against active attacks and to the initiator’s only against passive attacks. We start by presenting a simplified version of SIGMA-R without encryption:



The logic of the protocol is similar to that of the basic SIGMA. The difference is that B delays the sending of its identity and authentication information to the fourth message after it verified A ’s identity and authentication in message 3. This “similarity” in the logic of the protocol does not mean that its security is implied by that of the 3-message variants. Indeed, the protocol as described above is open to a *reflection attack* that is not possible against the 3-message variant. Due to the full symmetry of the protocol an attacker can simply replay each of the messages sent by A back to A . If A is willing to accept a key exchange with itself then A would successfully complete the protocol.⁷ Therefore, to prevent this attack the protocol needs to ensure some “sense of direction” in the authenticated

⁷ The only damage of this attack seems to be that it forces A to use a key derived from the distribution g^{x^2} rather than g^{xy} . These distributions may be distinguishable depending on the DH groups.

information. This can be done by explicitly adding different “tags” under the MAC for each of the parties (e.g., A would send $\text{MAC}_{K_m}(\text{“0”}, A)$ while B would send $\text{MAC}_{K_m}(\text{“1”}, B)$), or by using different MAC keys in each direction (i.e., instead of deriving a single key K_m from g^{xy} one would derive two keys, K_m and K'_m , where the former is used by A to compute its MAC and the latter by B). Any of these measures are sufficient to prevent the reflection attack and make the protocol secure [7] (another defense is for A to check that the peer’s DH exponential is different than her own.)

The full protocol SIGMA-R (with identity protection) is obtained by encrypting the last two messages in the above depicted protocol (and adding a reflection defense as discussed before). A “full fledge” illustration of protocol SIGMA-R is presented in [24].

Remark (the *inter-changeability property of SIGMA*). It is worth noting that the last two messages in the above protocol can be interchanged. Namely, B may proceed as described in SIGMA-R and wait for the reception of A ’s message (message 3 in the above picture) before sending his last message. But B may also decide to send his last message (signature and mac) immediately after, or together with, message 2 (which results in SIGMA-I). In this way, B may control if he is interested in protecting his own identity from active attacks or if he prefers to favor a faster exchange. The protocol may also allow for messages 3 and 4 to cross in which case the protocol is still secure but both identities may be open to active attacks.

5.4 Further Variants and the Use of SIGMA in IKE

As seen above the MAC of the sender’s identity is essential for SIGMA’s security. Here we present a variant of the protocol that differs from the above descriptions by the way the MAC value is placed in the protocol’s messages. Specifically, the idea is to include the MAC value under the signature (i.e., as part of the signed information). The interest on this variant is that it saves in message length by avoiding explicit sending of the MAC value, and more significantly because it is the variant of SIGMA adopted into the IKE protocol (both IKE version 1 [11] and version 2 [16]).

The MAC moved under the signature may cover just the identity of the sender or the whole signed information. For example, in B ’s message the pair $(\text{SIG}_B(g^x, g^y), \text{MAC}_{K_m}(B))$ is replaced with either (i) $\text{SIG}_B(g^x, g^y, \text{MAC}_{K_m}(B))$ or (ii) $\text{SIG}_B(\text{MAC}_{K_m}(g^x, g^y, B))$. In this way the space for an extra MAC outside the signature is saved, and the verification of the MAC is merged with that of the signature. In either case, *as long as the MAC covers the identity of the signer* then the same security of the basic SIGMA protocol (as well as SIGMA-I and SIGMA-R) is preserved⁸ [7]. Variant (ii) is used in the IKE protocol (version

⁸ A technicality here is that moving the MAC inside is possible only for MAC functions whose verification is done by recomputation of the MAC value; this is the case for almost all practical MAC functions, in particular when the MAC is implemented via a pseudorandom function as in IKE [11,16].

1) [11] in two of its authentication modes: the signature-based exchange of IKE uses the basic 3-message SIGMA protocol (without identity encryption) as presented in Section 5.1 for its aggressive mode, and it uses the 4-message SIGMA-R in its main mode. (In the later case, the use of SIGMA-R in IKE is preceded by two extra messages for negotiating security parameters.) In IKE the MAC function is implemented via a pseudorandom function which is also used in the protocol for the purpose of key expansion and derivation.⁹ IKE version 2 [16] uses variant (i) with SIGMA-R as its single key exchange method authenticated with public keys. In this protocol the peer’s DH exponential is not signed; the essential freshness guarantee is provided by signing a nonce chosen by the peer (see Section 5.1).

The SIGMA-R protocol has also been adopted in the JFK protocol [1] which has been proposed in the context of the undergoing revision of the IKE protocol. We note that in both [16,1] protocol SIGMA-R is augmented with mechanisms that provide some defense against Denial-of-Service attacks as discussed in Section 2.3.

Acknowledgment. I wish to thank Ran Canetti for embarking with me on a long and challenging journey towards formalizing and proving the security of key-exchange protocols (in particular, the SIGMA protocols). Special thanks to Dan Harkins for being receptive to my design suggestions when specifying the signature modes of IKE, and to Charlie Kaufman (and the IPsec WG) for incorporating this design into IKEv2. Thanks to Sara Bitan and Paul Van Oorschot for many useful discussions, and to Pau-Chen Cheng for sharing much of his implementation and system experience with me. This research was partially funded by the Irwin and Bethea Green & Detroit Chapter Career Development Chair.

References

1. B. Aiello, S. Bellovin, M. Blaze, R. Canetti, J. Ioannidis, A. Keromytis, O. Reingold, “Efficient, DoS-Resistant Secure Key Exchange for Internet Protocols,” *ACM Computers and Communications Security conference (CCS)*, 2002.
<http://www.research.att.com/~smb/papers/jfk-ccs.pdf>
2. M. Bellare and P. Rogaway, “Entity authentication and key distribution”, *Advances in Cryptology, - CRYPTO’93*, Lecture Notes in Computer Science Vol. 773, D. Stinson ed, Springer-Verlag, 1994, pp. 232–249.
3. S. M. Bellovin, “Problem Areas for the IP Security Protocols”, *Proceedings of the Sixth Usenix Unix Security Symposium*, 1996.
4. S. Blake-Wilson and A. Menezes, “Unknown key-share attacks on the station-to-station (STS) protocol”, *Proceedings of PKC ’99*, Lecture Notes in Computer Science, 1560 (1999), 154–170.

⁹ This use of a prf –as a MAC – under the signature has been a source of confusion among analysts of the IKE protocol; the prf was sometimes believed to have some other functionality related to the signature. It is important then to realize that its functionality under the signature is simply (and essentially!) that of a MAC covering the signer’s identity.

5. M. Burrows, M. Abadi and R. Needham, "A logic for authentication," *ACM Trans. Computer Systems* Vol. 8 (Feb. 1990), pp. 18–36
6. Canetti, R., and Krawczyk, H., "Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels", Eurocrypt'2001, LNCS Vol. 2045. Full version in: *Cryptology ePrint Archive* (<http://eprint.iacr.org/>), Report 2001/040.
7. Canetti, R., and Krawczyk, H., "Security Analysis of IKE's Signature-based Key-Exchange Protocol", *Crypto 2002*. LNCS Vol. 2442. Full version in: *Cryptology ePrint Archive* (<http://eprint.iacr.org/>), Report 2002/120.
8. W. Diffie, P. van Oorschot and M. Wiener, "Authentication and authenticated key exchanges", *Designs, Codes and Cryptography*, 2, 1992, pp. 107–125. Available at <http://www.scs.carleton.ca/~paulv/papers/sts-final.ps>.
9. N. Ferguson and B. Schneier, "A Cryptographic Evaluation of IPsec", <http://www.counterpane.com/ipsec.html>, 1999.
10. O. Goldreich, "Foundations of Cryptography: Basic Tools", Cambridge Press, 2001.
11. D. Harkins and D. Carrel, ed., "The Internet Key Exchange (IKE)", *RFC 2409*, Nov. 1998.
12. ISO/IEC IS 9798-3, "Entity authentication mechanisms — Part 3: Entity authentication using asymmetric techniques", 1993.
13. C. Jutla, "Encryption Modes with Almost Free Message Integrity", *Advances in Cryptology – EUROCRYPT 2001 Proceedings*, Lecture Notes in Computer Science, Vol. 2045, Springer-Verlag, B. Pfitzmann, ed, 2001.
14. Karn, P., and Simpson W.A., "The Photuris Session Key Management Protocol", draft-ietf-ipsec-photuris-03.txt, Sept. 1995.
15. B. Kaliski, "An unknown key-share attack on the MQV key agreement protocol", *ACM Transactions on Information and System Security (TISSEC)*. Vol. 4 No. 3, 2001, pp. 275–288.
16. C. Kaufman, "Internet Key Exchange (IKEv2) Protocol", draft-ietf-ipsec-ikev2-07.txt, April 2003 (to be published as an RFC).
17. S. Kent and R. Atkinson, "Security Architecture for the Internet Protocol", *Request for Comments 2401*, Nov. 1998.
18. S. Kent and R. Atkinson, "IP Encapsulating Security Payload (ESP)", *Request for Comments 2406*, Nov. 1998.
19. H. Krawczyk, Communication to IPsec WG, *IPsec mailing list archives*, April-October 1995. <http://www.vpnc.org/ietf-ipsec/>
20. H. Krawczyk, "SKEME: A Versatile Secure Key Exchange Mechanism for Internet," , *Proceedings of the 1996 Internet Society Symposium on Network and Distributed System Security*, Feb. 1996, pp. 114–127. <http://www.ee.technion.ac.il/~hugo/skeme-lncs.ps>
21. Krawczyk, H., Bellare, M., and Canetti, R., "HMAC: Keyed-Hashing for Message Authentication", *RFC 2104*, February 1997.
22. Krawczyk, H., "Blinding of Credit Card Numbers in the SET Protocol", *Proceedings of Financial Cryptography'99*, LNCS Vol. 1648, 1999.
23. H. Krawczyk, "The order of encryption and authentication for protecting communications (Or: how secure is SSL?)", *Crypto'2001*, LNCS Vol. 2139. Full version in: *Cryptology ePrint Archive* (<http://eprint.iacr.org/>), Report 2001/045.
24. H. Krawczyk, "SIGMA: the 'SIGn-and-Mac' Approach to Authenticated Diffie-Hellman and its Use in the IKE Protocols", full version. <http://www.ee.technion.ac.il/~hugo/sigma.html>
25. G. Lowe, "Some New Attacks upon Security Protocols", *9th IEEE Computer Security Foundations Workshop*, IEEE Press 1996, pp.162–169.

26. Meadows, C., “Analysis of the Internet Key Exchange Protocol Using the NRL Protocol Analyzer”, *Proc. of the 1999 IEEE Symposium on Security and Privacy*, IEEE Computer Society Press, 1999.
27. A. Menezes, P. Van Oorschot and S. Vanstone, “Handbook of Applied Cryptography,” CRC Press, 1996.
28. Orman, H., “The OAKLEY Key Determination Protocol”, *Request for Comments 2412*, Nov. 1998.
29. Perlman, R., and Kaufman, C., “Analysis of the IPsec key exchange Standard”, *WET-ICE Security Conference*, MIT, 2001.
30. V. Shoup, “On Formal Models for Secure Key Exchange”, Theory of Cryptography Library, 1999. Available at:
<http://philby.ucsd.edu/cryptolib/1999/99-12.html>.
31. P. van Oorschot, “Extending cryptographic logics of belief to key agreement protocols”, *Proceedings, 1st ACM Conference on Computer and Communications Security*, Nov. 1993, Fairfax, Virginia, pp. 232–243,